Aufgabe: Ein bestehendes Projekt lesen und erweitern

Für: Michael
Projekt: marinaaol/BankAccountManagementSystem (C#-Konsolenprojekt, ca. 230 Zeilen, Namen auf Portugiesisch)
Bearbeitungszeit: ca. 3-4 Stunden, verteilt auf mehrere Tage
Voraussetzung: Du kennst Vererbung, Polymorphie, Abstraktion, Interfaces, Enums und Exceptions aus den Einzelaufgaben.
Lernziel: Du wendest die Konzepte nicht auf der grünen Wiese an, sondern in **fremdem Code**. Das ist der Alltag im Beruf: Code verstehen, einen sinnvollen Plan machen, Schritt für Schritt erweitern, ohne alles kaputt zu machen.

> **Wichtig:** Das Projekt gehört einer anderen Person und hat keine Lizenzdatei. Du darfst es **lokal zum Lernen** klonen und verändern. Veröffentliche deine Version nicht als dein eigenes Werk.

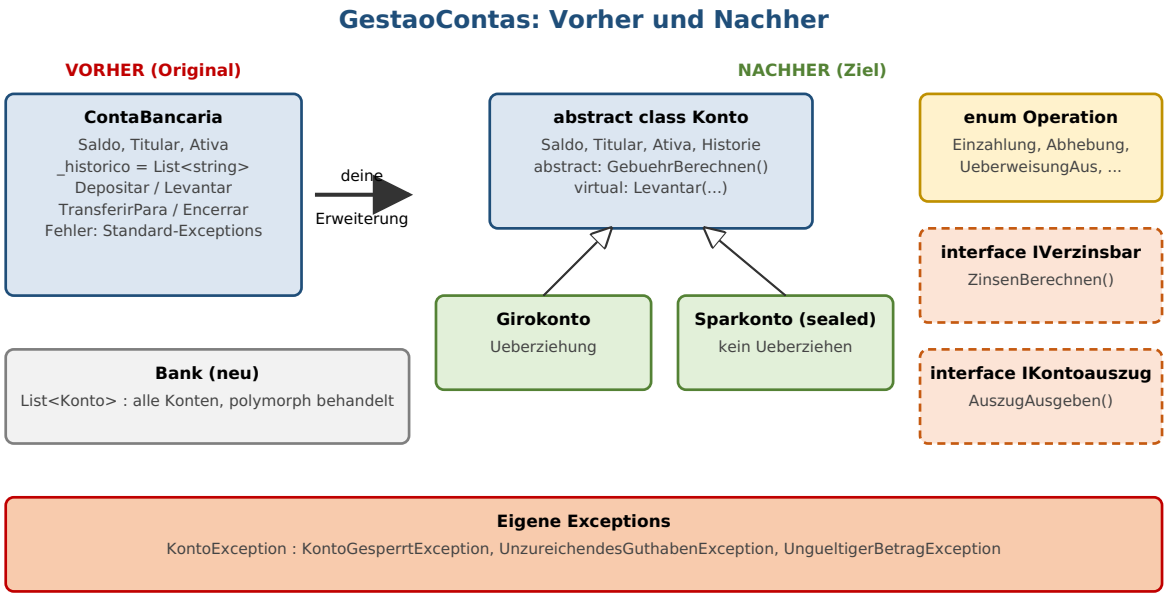
---

1. Das Projekt in Kürze

Ein kleines Programm für Bankkonten. Es gibt **eine** Klasse `ContaBancaria` und eine `Program.cs` mit Testszenarien.

Portugiesisch	Deutsch
Conta	Konto
Titular	Kontoinhaber
Saldo	Kontostand
Depositar	einzahlen
Levantar	abheben
TransferirPara	überweisen an
LimiteDescoberto	Überziehungslimit
Ativa / Encerrar	aktiv / schliessen
Historico / Extrato	Verlauf / Kontoauszug

2. Bild: Vorher und Nachher



Vorher und Nachher

Fragen zum Bild (vor dem Programmieren beantworten):

1. Welche der Konzepte aus unseren Aufgaben fehlen im Original noch ganz?
2. Warum könnte man einen **Verlauf aus Texten** (`List<string>`) später bereuen?
3. Warum steht `Konto` im Ziel als `abstract`? Welche Konten kann man im echten Leben “einfach so” eröffnen?
4. Welche Konten brauchen unterschiedliche Regeln beim Abheben?

---

3. Stufen

> **Regel:** Arbeite Stufe für Stufe. Nach **jeder** Stufe muss das Projekt kompilieren und die bestehenden Tests in `Program.cs` müssen noch laufen. Schreibe vor jeder Stufe auf, **was** du ändern willst und **warum**. Mache nach jeder Stufe einen Commit.

Stufe 0: Einrichten und Code lesen (ca. 30 Min.)

1. Klone das Projekt in einen eigenen Ordner und starte es. Wie viele Zeilen Ausgabe erwartest du? Sage es vorher voraus.
2. Lies `ContaBancaria.cs` **Zeile für Zeile**. Markiere mit drei Farben:
   - Grün: Dinge, die du schon verstehst und aus unseren Aufgaben kennst
   - Gelb: Dinge, die du nur teilweise verstehst
   - Rot: Dinge, die neu sind (z.B. `nameof`, `=>`, mehrere Konstruktoren)
3. Beantworte schriftlich:
   - Welche Felder und Properties gibt es? Welche sind nur lesbar von aussen?
   - Welche Prüfungen kommen in **mehreren** Methoden vor?
   - Welche Exception-Typen werden verwendet, und wo?
   - Wo wird der Verlauf geschrieben, und wie wird er angezeigt?
   - Was passiert mit dem Verlauf bei einer Überweisung? Findest du das elegant? Begründe.
4. Zeichne den Ablauf von `TransferirPara` als Aufrufkette: Welche Methoden rufen sich gegenseitig auf?

Stufe 1: Enum statt Text im Verlauf

Der Verlauf besteht aus Zeichenketten wie “Depósito de … EUR”.

- Definiere ein Enum `TipoOperacao` mit sinnvollen Werten (Einzahlung, Abhebung, Überweisung gesendet, Überweisung erhalten).
- Lege eine kleine Klasse `Operacao` an mit Typ, Betrag, Zeitpunkt und optionalem Text.

- Ersetze `List<string>` durch `List<Operacao>`.
- Passe `MostrarExtrato()` an. Der Text für jeden Typ entsteht in einem `switch`.

Überlege: Welche neue Auswertung wird jetzt möglich, die mit Texten schwierig war (z.B. “Summe aller Einzahlungen”)? Setze sie um.

Stufe 2: Eigene Exceptions

Das Projekt wirft `InvalidOperationException` für ganz verschiedene Fälle.

- Lege `KontoException : Exception` an plus drei Unterklassen: `KontoGesperrtException`, `UnzureichendesGuthabenException`, `UngueltigerBetragException`.
- Ersetze die passenden `throw`-Stellen. Wo bleibt eine Standard-Exception sinnvoll? Begründe jede Entscheidung.
- Passe `Program.cs` an: Fange die Fälle **gezielt** und gib verständliche Meldungen aus.
- Ergänze ein `finally` in einem Test. Was soll dort passieren?

Stufe 3: Wiederholungen entfernen

Die Prüfung “Konto aktiv?” steht in mehreren Methoden.

- Verschiebe sie in eine **private Methode** und rufe sie überall auf.
- Das Gleiche für die Prüfung “Betrag positiv”.
- Alle Tests müssen unverändert laufen. Warum ist das wichtig?

Stufe 4: Abstrakte Basisklasse und Vererbung

- Mache aus `ContaBancaria` eine `abstract class Konto` (deutsche oder portugiesische Namen: bleibe konsequent).
- Lege `Girokonto` (darf überziehen) und `Sparkonto` (darf **nicht** überziehen, `sealed`) an.
- Ergänze eine abstrakte Methode `GebuehrBerechnen()` mit unterschiedlicher Logik pro Konto.
- Das Abheben prüft das Limit über eine `virtual` Methode, die das Sparkonto überschreibt.
- Passe die Tests an: Statt `new ContaBancaria(...)` erzeugst du jetzt konkrete Kontoarten.

Überlege: Welche Teile bleiben in der Basisklasse, welche wandern in die Kindklassen? Notiere deine Begründung.

Stufe 5: Interfaces

- `IVerzinsbar` mit `ZinsenBerechnen()`: Nur das Sparkonto setzt es um.
- `IKontoauszug` mit `AuszugAusgeben()`: Alle Konten setzen es um.
- Schreibe eine Methode `Jahresabschluss(IEnumerable<IVerzinsbar> konten)`, die für alle verzinsbaren Konten die Zinsen gutschreibt.

Überlege: Warum ist `IVerzinsbar` ein Interface und keine Methode in `Konto`?

Stufe 6: Polymorphie mit einer Bank

- Lege eine Klasse `Bank` mit `List<Konto>` an.
- Methoden: `KontoHinzufuegen`, `Ueberweisen(nummerVon, nummerNach, betrag)`, `Uebersicht()`.
- In `Bank` darfst du **nirgends** prüfen, welche konkrete Kontoart ein Konto hat (kein `is`, kein `as`). Wenn du es doch brauchst: Notiere warum und ob es einen besseren Weg gäbe.
- `Uebersicht()` gibt mit `foreach` jedes Konto aus. Der Text kommt aus `ToString()` bzw. der überschriebenen Version.

Stufe 7: Zusatz (mind. eine Erweiterung wählen)

- Menü mit Eingabe und robuster Fehlerbehandlung (`TryParse`)
- Konto sperren mit `enum Kontostatus` statt `bool Ativa`
- Neues `Studentenkonto` mit eigener Gebührenregel, **ohne** `Bank` zu ändern
- Geld mit `decimal` statt `double` führen. Informiere dich vorher, warum man das in Banken so macht, und notiere es.

---

4. Bug-Labor

Baue jeden Fehler einzeln ein, **sage die Meldung vorher voraus**, starte, und trage das Ergebnis ein. Danach den Fehler wieder entfernen.

Nr.	Experiment	Meine Vorhersage	Meldung in eigenen Worten	Was habe ich gelernt?
1	`new Konto(...)` bei abstrakter Klasse			
2	In `Sparkonto` die abstrakte Methode nicht überschreiben			
3	`class Spezialkonto : Sparkonto`			
4	`TipoOperacao t = "Einzahlung";`			
5	`catch (Exception)` vor `catch (UnzureichendesGuthabenException)`			
6	Auf einem `Konto` die Methode `ZinsenBerechnen()` aufrufen			
7	Betrag `0` einzahlen, ohne `try`			
8	`Sparkonto` setzt `IVerzinsbar` um, aber `ZinsenBerechnen` fehlt			

5. Abnahmetests

Hake ab, sobald dein Programm das tut. Schreibe zu jedem Test auf, was du **getan** hast und was **passiert** ist.

- ☐ Alle ursprünglichen Tests aus `Program.cs` laufen weiterhin (oder sind bewusst angepasst, mit Begründung).
- ☐ Ein Sparkonto kann nicht überzogen werden. Die Meldung kommt von der **eigenen** Exception.
- ☐ Ein Girokonto kann bis zum Limit überzogen werden.
- ☐ Der Kontoauszug zeigt Beträge und Typen, ohne dass irgendwo ein Text zusammengebastelt wurde, der später ausgewertet werden müsste.
- ☐ `Bank.Uebersicht()` zeigt alle Konten, ohne dass die Bank die Kontoart kennt.
- ☐ `Jahresabschluss` verarbeitet nur verzinsbare Konten.
- ☐ Ein gesperrtes Konto lehnt jede Operation ab.
- ☐ Die Prüfung “aktiv” steht nur noch **einmal** im Code.

6. Erklär-Check

Erkläre Jasmin **mit Hilfe deiner Zeichnungen**, ohne in den Code zu schauen:

1. Welche Konzepte hast du wo eingebaut, und warum gerade dort?
2. Was war im fremden Code leicht zu verstehen, was schwer?
3. Warum ist `Konto` abstrakt, `Sparkonto` aber `sealed`?
4. Wo siehst du Polymorphie in deiner Version? Was passiert bei `konto.Levantar(...)` für ein `Sparkonto` und für ein `Girokonto`?
5. Warum ist ein Enum im Verlauf besser als ein Text?
6. Warum sind eigene Exception-Klassen besser als `InvalidOperationException` für alles?
7. Welche Änderung am Original war riskant? Wie hast du sichergestellt, dass nichts kaputt geht?
8. Was würdest du als Nächstes verbessern?

7. Bewertungsraster (Ampel)

Bereich	Selbst	Jasmin	Notiz
Fremden Code gelesen und verstanden (Stufe 0)			
Enum sinnvoll eingesetzt			
Eigene Exceptions und gezieltes Fangen			
Wiederholungen entfernt (kleine Methoden)			
Abstrakte Klasse, Vererbung, `sealed`			
`virtual` / `override` und Polymorphie			
Interfaces sinnvoll eingesetzt			
Bestehende Tests nicht kaputt gemacht			
Commits mit klaren Beschreibungen			
Erklär-Check in eigenen Worten			

Ziel: Mindestens Gelb in jeder Zeile, Grün bei Vererbung, Polymorphie und Exceptions.

8. Wie geht es weiter?

- Tests als echte Unit-Tests (xUnit) statt Ausgaben in `Program.cs`
- C# Files: Konten und Verlauf in einer Datei speichern